NAME: HARI RISHIKA KOMATI

ENROLLMENT NUMBER: 2403A51083

COURSE: AIDL

ASSIGNMENT: 6

TASK:

Lab/Exp 5: Applying Optimizers -Adam, SGD and Observing Convergence.

Kaggle Dataset Link: https://www.kaggle.com/c/cifar-10

Tasks:

1. **Load and preprocess the CIFAR-10 dataset/example.**
2. **Build the required PyTorch model/program.**
3. **Train/execute the model.**
4. **Evaluate using suitable metrics.**
5. **Visualize results using plots where applicable.**
6. **Write a brief conclusion comparing observations.**

```python
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])
train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
```

```
            self.classifier = nn.Sequential(
                nn.Flatten(),
                nn.Linear(64 * 8 * 8, 128),
                nn.ReLU(),
                nn.Linear(128, 10)
            )
        def forward(self, x):
            x = self.features(x)
            x = self.classifier(x)
            return x
    def train_model(optimizer_name, epochs=10):
        model = CNN()
        criterion = nn.CrossEntropyLoss()
        if optimizer_name == "SGD":
            optimizer = optim.SGD(
                model.parameters(),
                lr=0.01,
                momentum=0.9
            )

        elif optimizer_name == "Adam":
            optimizer = optim.Adam(
                model.parameters(),
                lr=0.001
            )
        loss_history = []
        accuracy_history = []
        for epoch in range(epochs):
            model.train()
            running_loss = 0.0
            correct = 0
            total = 0
            for images, labels in train_loader:
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                running_loss += loss.item()
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
            epoch_loss = running_loss / len(train_loader)
            epoch_accuracy = 100 * correct / total
            loss_history.append(epoch_loss)
```

```
        loss_history.append(epoch_loss)
        accuracy_history.append(epoch_accuracy)
        print(
            f"{optimizer_name} - Epoch "
            f"{epoch + 1}/{epochs} "
            f"Loss: {epoch_loss:.4f} "
            f"Accuracy: {epoch_accuracy:.2f}%"
        )
    return model, loss_history, accuracy_history
print("\nTraining with SGD...\n")
sgd_model, sgd_loss, sgd_accuracy = train_model(
    "SGD",
    epochs=10
)
print("\nTraining with Adam...\n")
adam_model, adam_loss, adam_accuracy = train_model(
    "Adam",
    epochs=10
)
def evaluate_model(model):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    return accuracy
sgd_test_accuracy = evaluate_model(sgd_model)
adam_test_accuracy = evaluate_model(adam_model)
print("\nFinal Test Accuracy")
print("--------------------")
print(f"SGD  : {sgd_test_accuracy:.2f}%")
print(f"Adam : {adam_test_accuracy:.2f}%")
plt.figure(figsize=(8, 5))
plt.plot(sgd_loss, label="SGD")
plt.plot(adam_loss, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("SGD vs Adam - Training Loss")
plt.legend()
plt.grid()
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(sgd_accuracy, label="SGD")
plt.plot(adam_accuracy, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Training Accuracy (%)")
plt.title("SGD vs Adam - Training Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
100%|██████████| 170M/170M [33:54<00:00, 83.8kB/s]

Training with SGD...

SGD - Epoch 1/10 Loss: 1.5407 Accuracy: 43.89%
SGD - Epoch 2/10 Loss: 1.0984 Accuracy: 61.24%
SGD - Epoch 3/10 Loss: 0.9033 Accuracy: 68.12%
SGD - Epoch 4/10 Loss: 0.7715 Accuracy: 72.89%
SGD - Epoch 5/10 Loss: 0.6538 Accuracy: 77.12%
SGD - Epoch 6/10 Loss: 0.5436 Accuracy: 80.88%
SGD - Epoch 7/10 Loss: 0.4457 Accuracy: 84.27%
SGD - Epoch 8/10 Loss: 0.3563 Accuracy: 87.40%
SGD - Epoch 9/10 Loss: 0.2684 Accuracy: 90.66%
SGD - Epoch 10/10 Loss: 0.2059 Accuracy: 92.76%

Training with Adam...

Adam - Epoch 1/10 Loss: 1.3258 Accuracy: 52.34%
Adam - Epoch 2/10 Loss: 0.9576 Accuracy: 66.13%
Adam - Epoch 3/10 Loss: 0.8095 Accuracy: 71.47%
Adam - Epoch 4/10 Loss: 0.6993 Accuracy: 75.53%
Adam - Epoch 5/10 Loss: 0.5946 Accuracy: 79.26%
Adam - Epoch 6/10 Loss: 0.5014 Accuracy: 82.47%
Adam - Epoch 7/10 Loss: 0.4130 Accuracy: 85.52%
Adam - Epoch 8/10 Loss: 0.3316 Accuracy: 88.30%
Adam - Epoch 9/10 Loss: 0.2590 Accuracy: 91.02%
Adam - Epoch 10/10 Loss: 0.2036 Accuracy: 92.88%
```

```
Final Test Accuracy
--------------------
SGD  : 70.69%
Adam : 71.82%
```